Lecture 3

More Typescript

Arrays Demo

Lambdas (Anonymous Functions)

- Often used in conjunction with higher-order functions.
  - e.g. `map()`, `forEach()`, `reduce()` on arrays

- Closures
  - Anonymous functions can refer to externally defined variables.
  - The runtime environment ensures that the “transitive closure” of everything it uses is preserved in memory as long as needed.

Strings Demo

refer to docs

data.reverse() // change in place (arrays mutable)

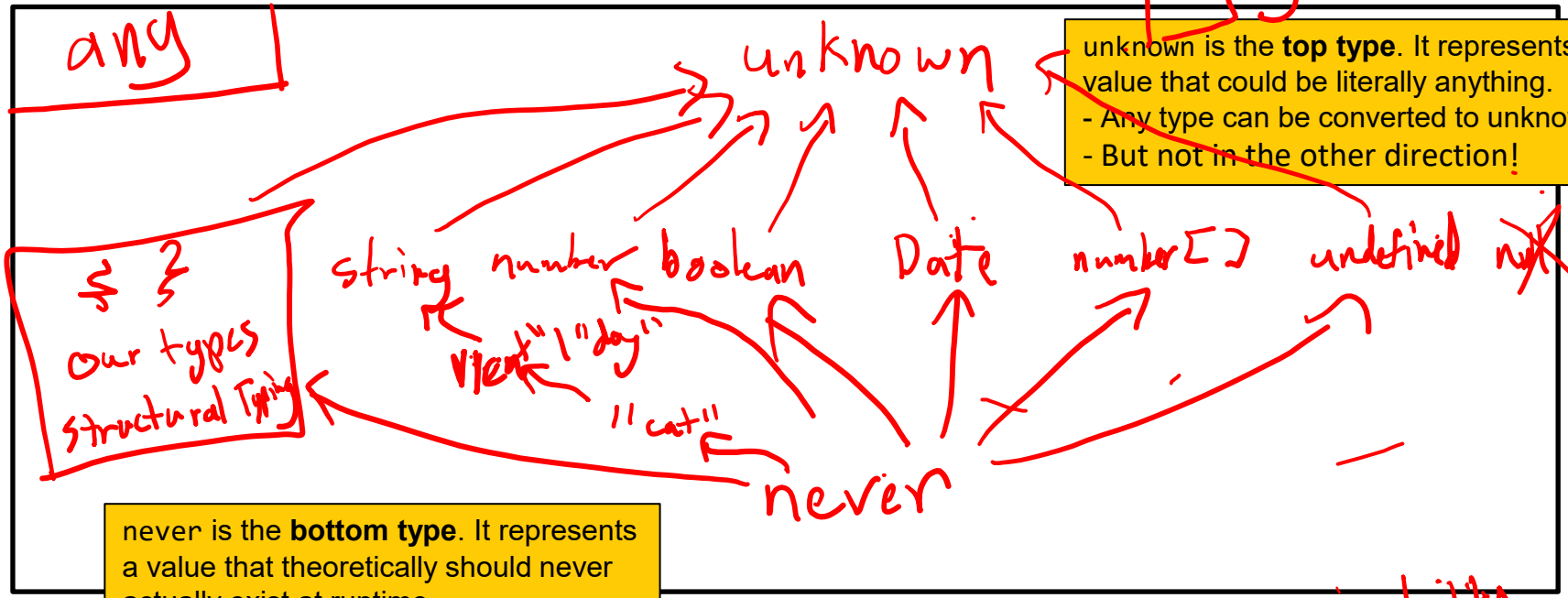
str.replace( ) // return adjusted as new string

Documentation and Inquiry

- MDN JavaScript Docs
  (Individual pages usually show up in web searches)
  https://developer.mozilla.org/en-US/docs/Web/JavaScript

- Generative AI Tools
  Focus on Interactive Dialog
  Don’t settle for not understanding it yourself!

TypeScript Type Lattice



Interfaces and Type Aliases

- `interface` Definitions

```
interface Circle {
    readonly name: string;
    radius: number;
}
```

- `type` Aliases

```
type Circle = {
    readonly name: string;
    radius: number;
};
```

- `readonly` modifier
- Prefer `interface` unless you need the features of `type`.

Type Assertions (i.e. explicit type casts)

- Two syntax options
  - `<DestinationType> original`
  - `original as DestinationType`

- The latter form is preferred since it's less likely to interfere with other related web technologies that use < > syntax

Structural Typing

- Shape and assignability

```
interface Shape {
    name: string;
};

let s: Shape;
let cs: ColoredShape;

s = cs;
s.color
```

```
interface Human {
    name: string;
}

interface ColoredShape extends Shape {
    //name: string; // not needed
    color: string;
};

interface Circle {
    name: string;
    radius: number;
};
```

Literal Types

let n : 2;
n = 2;

let pet : "cat" | "dog" | "lizard";
x = "cat";
x = "dog"; // error

- A type may only be a single `number` or `string` value.
- Examples

Intersection and Union Types

Person & Robot

- Intersection Types
  - Use the & operator in the type definition
  - *"This **and** That":* You may use properties from either type.

Person | Robot
.talk()
.charge()

number | string
string | undefined
"cat" | "dog"

- Union Types (|)
  - Use the | operator in the type definition
  - *"This **or** That":* You can only use properties that both types have.

Discriminated Unions

- A union type with a discriminant property.
  - Check the discriminant on an object to know which variant it is.

```ts
type Shape = Circle | Square | Triangle;

interface Circle {
  kind: "circle";
  radius: number;
}

interface Square {
  kind: "square";
  side: number;
}

interface Triangle {
  kind: "triangle";
  a: number; b: number; c: number;
}
```

We return the result of `assertNever()` here to satisfy the `number` return type of `perimeter()`. Because it always throws at runtime, that's OK - and it matches the core idea of the never type.

```ts
function perimeter(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      return 2 * Math.PI * shape.radius;
    case "square":
      return 4 * shape.side;
    case "triangle":
      return shape.a + shape.b + shape.c;
    default:
      // Exhaustiveness check
      return assertNever(shape);
  }
}

function assertNever(x: never): never {
  throw new Error("Unexpected object: " + x);
}
```

Type narrowing allows the compiler to safely check access to variant-specific properties in each case.

`assertNever(shape)` gives a compiler error if we missed a case.

Ternary Operator : ?

General Form
`condition ? result_if_true : result_if_false`

- Use the ternary operator for an “if-else expression”

```
const age = 67;  // 15% senior discount
const discount = age >= 65 ? 0.15 : 0;


const is_bold = ...; // conditional css style
const html = `<p
  ${is_bold ? "style="font-weight: bold;" : ""}
>Hello, World!</p>`;


// similar to an if/else if structure
function letter_grade(score: number): string {
  return score >= 90 ? "A" :
         score >= 80 ? "B" :
         score >= 70 ? "C" :
         score >= 60 ? "D" :
         "E";
}
```

```
function perimeter(shape: Shape): number {
  return shape.kind === "circle" ? 2 * Math.PI * shape.radius :
         shape.kind === "square" ? 4 * shape.side :
         shape.kind === "triangle" ? shape.a + shape.b + shape.c :
         assertNever(shape); // Exhaustiveness check
}

function assertNever(x: never): never {
  throw new Error("Unexpected object: " + x);
}
```

Alternative approach to the switch statement on the previous slide. We use the same exhaustiveness check.

Optional Chaining and Nullish Coalescing

- Shortcut operators for common patterns

- **Optional Chaining**

  Access a property on an optional object (`undefined` if the object is not present)

```
obj?.some_prop;
```

```
// equivalent using ternary operator
obj !== null && obj !== undefined ? obj.some_prop : undefined;
```

- **Nullish Coalescing**

  Use a fallback value if the original is null or undefined.

```
value ?? fallbackValue
```

```
// equivalent using ternary operator
value !== null && value !== undefined ? value : fallbackValue;
```

Classes

- TypeScript supports most of the usual class features
  - Member variables (i.e. fields, properties)
  - Member functions (i.e. methods)
  - Constructors
  - Inheritance and Polymorphism
  - Instance vs. Static Members

Modules, Imports and Exports

- See project 1 specification for an explanation.